

ELEC 475 Lab 2 Report

Faculty of Engineering and Applied Science
Queen's University

Group 24

Nathan Daniel | 20289363
Tristan Aro Guevarra | 20353857

10/22/2026

Instructor: Michael Greenspan

2. Architectural Specifications

All four networks take a 3 × 64 × 64 RGB image and output K class scores (K = 10 for CIFAR-10 and K = 200 for Tiny ImageNet). The output width of each convolution or pooling layer is W_out = floor((W + 2p − k)/s) + 1 where W is the input width, k is the kernel size, s the stride and the padding is p. A convolution layer has C_out(C_in·k^2 + 1) parameters a fully connected layer has N_out(N_in + 1), and BatchNorm2d adds two per channel, which are activation, pooling, dropout and LRN layers have none. The receptive field is the size of the input region that one feature can see. It grows with each layer by (k − 1) times the total stride so far, starting from 1 pixel. Parameter counts are for K = 10 unless stated.

Base LeNet-5

The base model (models/lenet.py) follows LeNet-5 adapted to three channel input. The feature extractor has two convolution stages each consisting of a 5 × 5 convolution with a stride of 1 and no padding, a Tanh activation and 2 × 2 average pooling with stride 2. The first convolution maps 3 to 6 channels and the second maps 6 to 16 channels. The classifier flattens the feature map and applies three fully connected layers of 120, 84, and K units, with Tanh after the first two.

Layer dimensions: Conv1 reduces 64 to (64 − 5)/1 + 1 = 60, giving 6 × 60 × 60; pooling halves this to 6 × 30 × 30; Conv2 gives (30 − 5) + 1 = 26, or 16 × 26 × 26; and pooling gives 16 × 13 × 13. Flattening produces 16 × 13 × 13 = 2,704 features.

Receptive field: Conv1 gives r = 5 (j = 1); the first pool gives r = 5 + 1·1 = 6 (j = 2); Conv2 gives r = 6 + 4·2 = 14; and the second pool gives r = 14 + 1·2 = 16 (j = 4). Each classifier input therefore describes a 16 × 16 patch, a quarter of the image width.

Parameters: Conv1 has 6(3·25 + 1) = 456 and Conv2 has 16(6·25 + 1) = 2,416, so the convolutions hold only 2,872. FC1 has 120(2,704 + 1) = 324,600, FC2 has 84(120 + 1) = 10,164, and the output layer has K(84 + 1) = 850, giving a classifier of 335,614. The total is 338,486 (354,636 for K = 200), and FC1 alone holds 95.9% of the model.

Modernized LeNet

The modernized model (models/lenet_modern.py) keeps the same channel widths (6 and 16) and classifier sizes (120, 84, K) but makes the three changes tested in the in-lab session: Tanh is replaced with ReLU, 2 × 2 average pooling is replaced with overlapping 3 × 3 max pooling with stride 2 and each convolution is padded by 2 so that it preserves spatial size.

Layer dimensions: Conv1 keeps 64 × 64, since (64 + 4 − 5) + 1 = 64, giving 6 × 64 × 64; pooling gives (64 − 3)/2 + 1 = 31, or 6 × 31 × 31; Conv2 keeps 16 × 31 × 31; and pooling gives (31 − 3)/2 + 1 = 15, or 16 × 15 × 15. Flattening produces 3,600 features.

Receptive field: Conv1 gives r = 5 (j = 1); the 3 × 3 pool gives r = 5 + 2·1 = 7 (j = 2); Conv2 gives r = 7 + 4·2 = 15; and the second pool gives r = 15 + 2·2 = 19 (j = 4). The overlapping windows enlarge the receptive field from 16 to 19 pixels.

Parameters: the convolutions are unchanged at 2,872. Because the flattened input grows from 2,704 to 3,600, FC1 grows to 120(3,600 + 1) = 432,120, and the classifier totals 443,134. The model has 446,006 parameters (462,156 for K = 200); the entire increase of 107,520 over the base model is in FC1.

Full AlexNet-64

The full model (models/alexnet.py) implements the five-stage AlexNet adapted to 64 × 64 input, as specified in the handout. Because the input is much smaller than AlexNet's original 224 × 224, the first convolution uses a 5 × 5 kernel with stride 2 and padding 2 instead of 11 × 11 with stride 4. Stage 1 maps 3 to 64 channels, Stage 2 uses a 5 × 5 convolution (padding 2) to 192 channels Stages 3, 4, and 5 use 3 × 3 convolutions (padding 1) to 384, 384, and 256 channels. Stages 1, 2, and 5 end with overlapping 3 × 3 max pooling with stride 2, and every convolution is followed by ReLU. The classifier flattens the features and applies Dropout(p = 0.5), a 2,304 → 1,024 linear layer with ReLU, Dropout(p = 0.5), a 1,024 → 1,024 linear layer with ReLU, and a 1,024 → K output layer.

The --norm argument selects one of three normalization variants. With none, each stage is Conv → ReLU. With lrn, nn.LocalResponseNorm(size = 5, alpha = 10^{-4}, beta = 0.75, k = 2) follows the ReLU in Stages 1 and 2, as in the original AlexNet; it divides each activation by the summed squared activity of its five neighbouring channels, $b_c = a_c / (k + (\alpha/n)\Sigma_j a_j^2)^\beta$. With bn, nn.BatchNorm2d is placed between each convolution and its ReLU in all five stages; it normalizes each channel over the mini-batch and spatial dimensions, $\hat{x} = (x - \mu_B)/\sqrt{\sigma^2_B + \varepsilon}$, then applies a learned scale and shift $y = \gamma\hat{x} + \beta$, and uses running estimates of μ and σ^2 at evaluation time.

Layer dimensions: Conv1 gives (64 + 4 − 5)/2 + 1 = 32, or 64 × 32 × 32, and pooling gives (32 − 3)/2 + 1 = 15, or 64 × 15 × 15. Conv2 keeps 192 × 15 × 15 and pooling gives 192 × 7 × 7. Conv3, Conv4, and Conv5 keep the 7 × 7 size, producing 384 × 7 × 7, 384 × 7 × 7, and 256 × 7 × 7, and the final pool gives (7 − 3)/2 + 1 = 3, or 256 × 3 × 3. Flattening produces 256 × 3 × 3 = 2,304 features matching the handout.

Receptive field: Conv1 gives r = 5 (j = 2); pool 1 gives r = 5 + 2·2 = 9 (j = 4); Conv2 gives r = 9 + 4·4 = 25; pool 2 gives r = 25 + 2·4 = 33 (j = 8); Conv3, Conv4, and Conv5 each add 2·8 = 16, giving 49, 65, and 81; and pool 3 gives r = 81 + 2·8 = 97 (j = 16). A 97-pixel receptive field exceeds the 64-pixel input so every feature reaching the classifier can draw on the whole image.

Parameters: the convolutions hold 4,864 (Conv1), 307,392 (Conv2), 663,936 (Conv3), 1,327,488 (Conv4), and 884,992 (Conv5), a total of 3,188,672. The classifier holds 2,360,320 (FC1), 1,049,600 (FC2), and 10,250 (output), a total of 3,420,170. The none and LRN variants therefore have 6,608,842 parameters (6,803,592 for K = 200). BatchNorm adds 2 × (64 + 192 + 384 + 384 + 256) = 2,560, giving 6,611,402 (6,806,152 for K = 200). The model profile printed by train.py reports exactly these totals; the handout's value of about 6.3 M is a nominal scale. Although the convolutions hold only 48.2% of the parameters, they perform 98.4% of the model's 218.35 million multiply accumulates.

PyTorch Model Zoo AlexNet

The reference baseline (models/zoo.py) loads the standard torchvision AlexNet with random initialization (pretrained = False) and replaces its final layer, classifier, with a 4,096 → K linear layer.

The architecture is the original 224 × 224 design: an 11 × 11 stride-4 convolution to 64 channels, a 5 × 5 convolution to 192, three 3 × 3 convolutions to 384, 256, and 256, ReLU after every convolution, three 3 × 3 stride-2 max pools, AdaptiveAvgPool2d((6, 6)), and a classifier of 9,216 → 4,096 → 4,096 → K with Dropout(p = 0.5) before the first two linear layers. It has no normalization layers.

Layer dimensions: Conv1 gives (64 + 4 − 11)/4 + 1 = 15, or 64 × 15 × 15; pooling gives 64 × 7 × 7; Conv2 keeps 192 × 7 × 7; pooling gives 192 × 3 × 3; Conv3 to Conv5 keep 3 × 3; and the final pool gives (3 − 3)/2 + 1 = 1, or 256 × 1 × 1. At 64 × 64 input the spatial map is therefore reduced to a single position. AdaptiveAvgPool2d((6, 6)) then copies that one 256-value vector into all 36 cells of a 6 × 6 grid, so the 9,216 classifier inputs contain only 256 distinct values and no spatial layout.

Receptive field: Conv1 gives r = 11 (j = 4); pool 1 gives 19 (j = 8); Conv2 gives 19 + 4·8 = 51; pool 2 gives 67 (j = 16); Conv3 to Conv5 give 99, 131, and 163; and pool 3 gives 195 (j = 32), about three times the input size.

Parameters: the convolutions hold 23,296, 307,392, 663,936, 884,992, and 590,080, a total of 2,469,696 (4.3%). The classifier holds 37,752,832 (FC1), 16,781,312 (FC2), and 40,970 (output), a total of 54,575,114 (95.7%). The model has 57,044,810 parameters (57,823,240 for K = 200), 8.6 times as many as AlexNet-64, and FC1 alone accounts for 66.2% of them.

3. Empirical Results & Figures

All runs use Adam (learning rate 0.001 and weight decay 0.0001), batch size 128 and seed 42 unless stated. Accuracy and Macro F1 are final epoch validation values, Δ = training accuracy − validation accuracy in percentage.

Master Table 7.3a: Architecture and Computational Complexity

Models are profiled following the conventions of He et al and Howard et al: trainable parameters, multiply accumulates (MACs), FLOPs and batch size one latency. Table 7.3a is shown separately for each dataset because classifier sizes and epoch times differ, parameters and FLOPs are profiler outputs. Device labels on epoch times are explicit so that GPU and CPU epoch times are not read as architectural speedups.

Table 7.3a: Architectural configuration and computational complexity: CIFAR-10 (inference latency on RTX 3070 GPU)

Model Architecture	Conv Depth	Normalization & Regularization	Data Augmentation	Trainable Params (M)	FLOPs (MFLOPs)	Mean Epoch Time (s)	Inference Latency (ms)
1. Base LeNet-5	2	None	None	0.338	7.17	38.05 (CPU)	0.40
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446	9.19	43.24 (CPU)	0.44
3. AlexNet (No Norm)	5	Dropout (p = 0.5)	None	6.609	436.70	705.85 (CPU)	0.86
4. AlexNet + LRN	5	LRN + Dropout (p = 0.5)	None	6.609	436.70	831.86 (CPU)	1.36
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p = 0.5)	None	6.611	437.97	772.06 (CPU)	1.01
6. AlexNet + BN + Aug	5	BatchNorm + Dropout (p = 0.5)	Pure PyTorch Aug	6.611	437.97	522.56 (CPU)	1.01*
7. PyTorch Zoo AlexNet	5	PyTorch Zoo (AdaptiveAvgPool2d)	None	57.045	188.20	229.79 (CPU)	1.86

Table 7.3a: Architectural configuration and computational complexity: Tiny ImageNet (inference latency on CPU)

Model Architecture	Conv Depth	Normalization & Regularization	Data Augmentation	Trainable Params (M)	FLOPs (MFLOPs)	Mean Epoch Time (s)	Inference Latency (ms)
1. Base LeNet-5	2	None	None	0.355	7.20	69.83 (CPU)	0.59
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.462	9.22	69.96 (CPU)	0.75
3. AlexNet (No Norm)	5	Dropout (p = 0.5)	None	6.804	437.08	502.44 (CPU)	6.48
4. AlexNet + LRN	5	LRN + Dropout (p = 0.5)	None	6.804	437.08	870.72 (CPU)	10.49

Augmentation is applied only during training, so row 6 shares row 5's parameters, FLOPs, and inference latency.

Table 7.3b: Empirical predictive performance across datasets

Model Architecture	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F1 (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1 (%)
1. Base LeNet-5	53.16	53.11	10.30	8.03
2. Modernized LeNet	64.55	63.99	19.23	17.14
3. AlexNet (No Norm)	73.62	73.56	14.49	12.21
4. AlexNet + LRN	71.82	71.84	16.55	14.39
5. AlexNet + BatchNorm	77.27	77.47	30.63	29.01
6. AlexNet + BN + Aug	82.47	82.68	29.60	27.48
7. PyTorch Zoo AlexNet	67.49	67.28	0.50	0.00

Epoch budgets follow the handout: 5 for rows 1–2, 10 for rows 3–5, 15 for row 6 on CIFAR-10 and 10 on Tiny ImageNet and 15 for row 7.

Table 3.1: Measured values against the handout reference scales

Model	Params (M): reference	Params (M): measured	MFLOPs: reference	MFLOPs: measured
Base LeNet-5	~0.34	0.338	~4.8	7.17
Modernized LeNet	~0.45	0.446	~8.2	9.19
AlexNet-64	~6.31	6.609 (6.611 with BN)	~250–257	436.70
PyTorch Zoo AlexNet	~57.04	57.045	~310–315	188.20

Measured parameter counts match the reference scales confirming the layer shapes are correct. The FLOP counts differ from the nominal references but a hand count of AlexNet-64's multiply accumulates (218.35 M, i.e. 436.70 MFLOPs) matches the profiler exactly all of the handout's expected trends are observed.

Comparative validation-accuracy curves for all seven configurations were generated with the provided utility:

```
python -m utils.visualization --plot-ablation
```

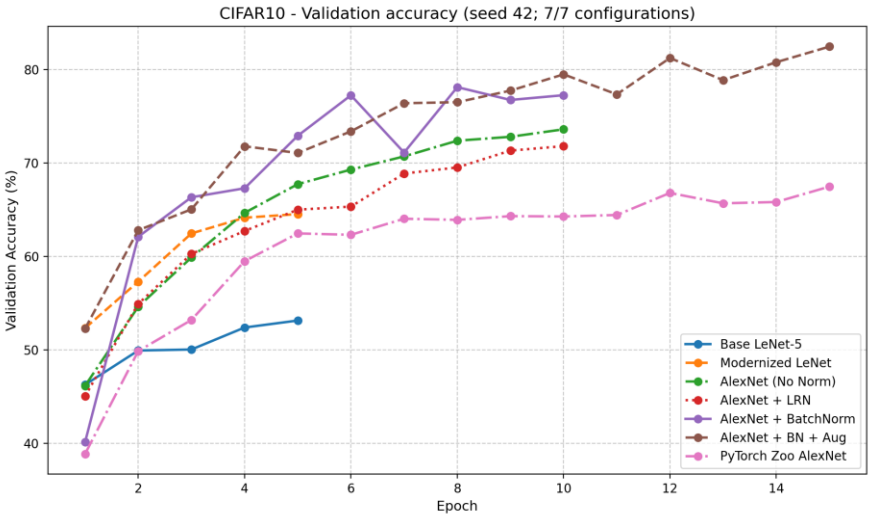


Figure 1: CIFAR-10 validation accuracy for the seven master configurations (seed 42).

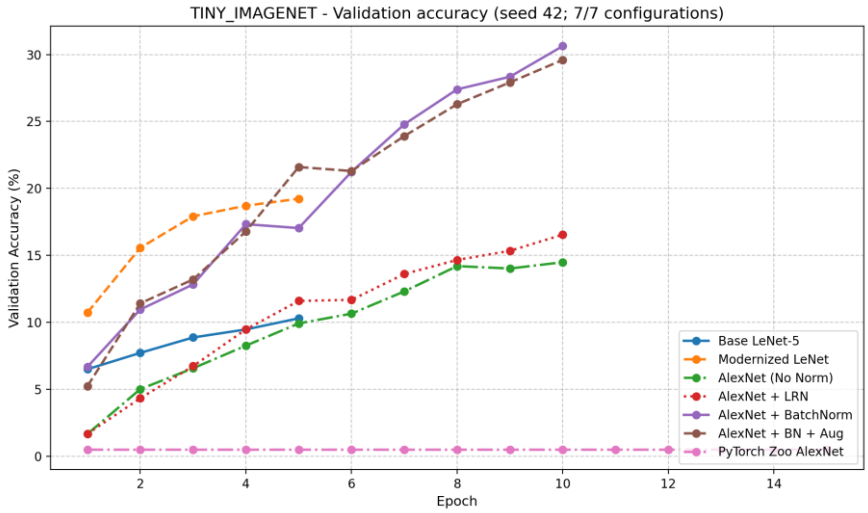


Figure 2: Tiny ImageNet validation accuracy for the seven master configurations (seed 42). The Zoo AlexNet remains at the 0.5% chance level.

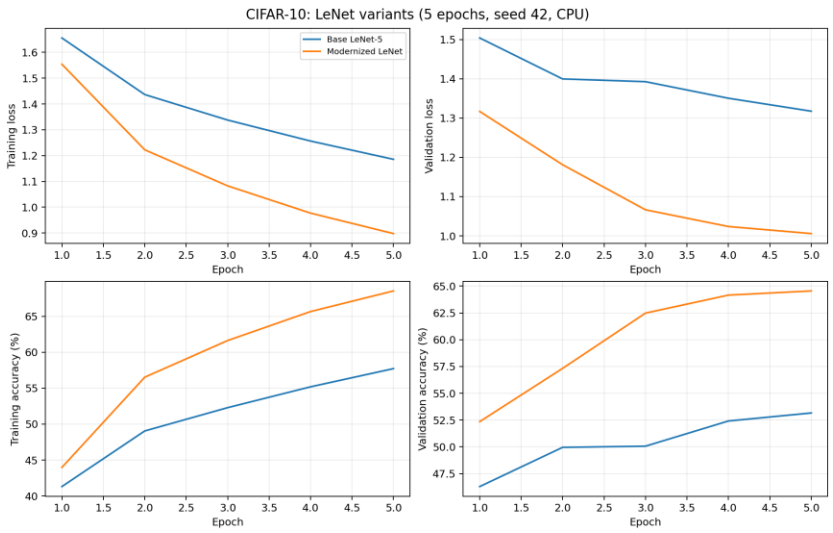


Figure 3: Training and validation loss (top) and accuracy (bottom) for base and modernized LeNet on CIFAR-10 (5 epochs).

Normalization: None vs LRN vs BatchNorm2d

The three AlexNet-64 variants were trained for 10 epochs on CIFAR-10. --no-augment was added to the handout commands so that normalization is the only difference between runs, LRN uses nn, LocalResponseNorm(size=5, alpha=1e-4, beta=0.75, k=2.0), BatchNorm uses nn.BatchNorm2d(num_features).

```
python train.py --model alexnet --norm none --dataset cifar10 --epochs 10 --no-augment --save-model
checkpoints/alexnet_none.pth --plot-file plots/loss_alexnet_none.png
```

```
python train.py --model alexnet --norm lrn --dataset cifar10 --epochs 10 --no-augment --save-model
checkpoints/alexnet_lrn.pth --plot-file plots/loss_alexnet_lrn.png
```

```
python train.py --model alexnet --norm bn --dataset cifar10 --epochs 10 --no-augment --save-model
checkpoints/alexnet_bn.pth --plot-file plots/loss_alexnet_bn.png
```

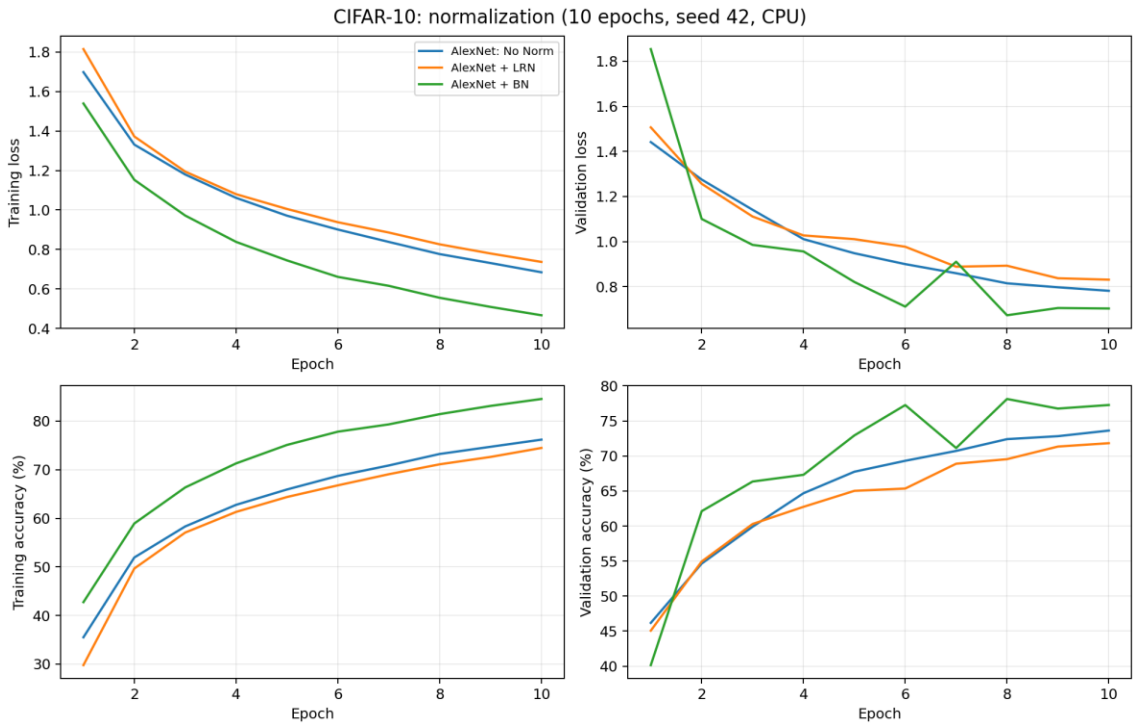


Figure 4: CIFAR-10 normalization study: training and validation loss (top) and accuracy (bottom).

Performance improvement - Yes. BatchNorm achieved 77.27% accuracy and 77.47% Macro F1 compared with 73.62% / 73.56% without normalization and 71.82% / 71.84% with LRN. The same ordering holds on Tiny ImageNet: 30.63% / 29.01% for BN versus 14.49% / 12.21% for None and 16.55% / 14.39% for LRN.

BatchNorm normalizes each channel over the mini-batch, $\hat{x} = (x - \mu_B)/\sqrt{\sigma^2_B + \varepsilon}$ then applies a learned scale and shift $y = \gamma\hat{x} + \beta$. Without it each weight updates in an earlier layer shifts the input distribution of every later layer forcing them to keep re-adapting fixing each channel's mean and variance removes most of that drift. It also smooths the loss landscape so gradients stay reliable over larger steps and the optimizer progresses faster. Consistent with this the BN model had the lowest training loss at every epoch.

Computational overhead - Yes. Both layers measurably increases cost but by very different amounts BN raised the profiled FLOPs from 436.70 to 437.97 M and added 2,560 parameters, the LRN adds no parameters and the profiler counts no FLOPs for it. Also the mean epoch time rose from 705.85 s (None) to 772.06 s with BN and 831.86 s with LRN and GPU inference latency rose from 0.86 ms to 1.01 ms (BN) and 1.36 ms (LRN). BN's element wise subtract and scale is negligible next to the convolutions whereas LRN's cross channel sum of squares and fractional power is memory bound and slow despite adding no counted FLOPs. BN's accuracy and faster convergence clearly justify its small cost LRN costs more and lowered accuracy consistent with the VGG finding.

Convergence stability - BatchNorm. All three training loss curves decrease monotonically but BN falls fastest and stays lowest at every epoch ending at 0.4668 versus 0.6845 (None) and 0.7369 (LRN) so it shows the most stable gradient progression (shown Figure 4 and top left). Validation loss is noisier for BN: it rose twice including a spike from 0.7115 to 0.9107 at epoch 7 that recovered to

0.6732 at epoch 8 while None decreased monotonically and LRN rose once. This volatility comes from the gap between per-batch statistics used in training and the running statistics used at evaluation at learning rate 0.001 and motivated the custom learning-rate experiment (Section 3.5).

Data augmentation and the generalization gap

utils/transforms.py applies Resize(64, 64), RandomCrop(64, padding=4, padding_mode="reflect"), RandomHorizontalFlip(p=0.5), and ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), followed by ToTensor and Normalize. AlexNet + BN (with dropout) was trained for 15 epochs on CIFAR-10 with and without augmentation:

```
python train.py --model alexnet --norm bn --dataset cifar10 --epochs 15 --no-augment --save-model
checkpoints/alexnet_no_aug.pth --plot-file plots/loss_no_aug.png
```

```
python train.py --model alexnet --norm bn --dataset cifar10 --epochs 15 --save-model
checkpoints/alexnet_with_aug.pth --plot-file plots/loss_with_aug.png
```

Table 3.2: Generalization gap with and without augmentation

Dataset	Augmentation	Train Acc (%)	Val Acc (%)	Val Macro F1 (%)	Gap Δ (pp)
CIFAR-10 (15 epochs)	Without	89.91	80.53	80.54	9.38
CIFAR-10 (15 epochs)	With	84.28	82.47	82.68	1.81
Tiny ImageNet (10 epochs)	Without	30.43	30.63	29.01	-0.20
Tiny ImageNet (10 epochs)	With	27.14	29.60	27.48	-2.46

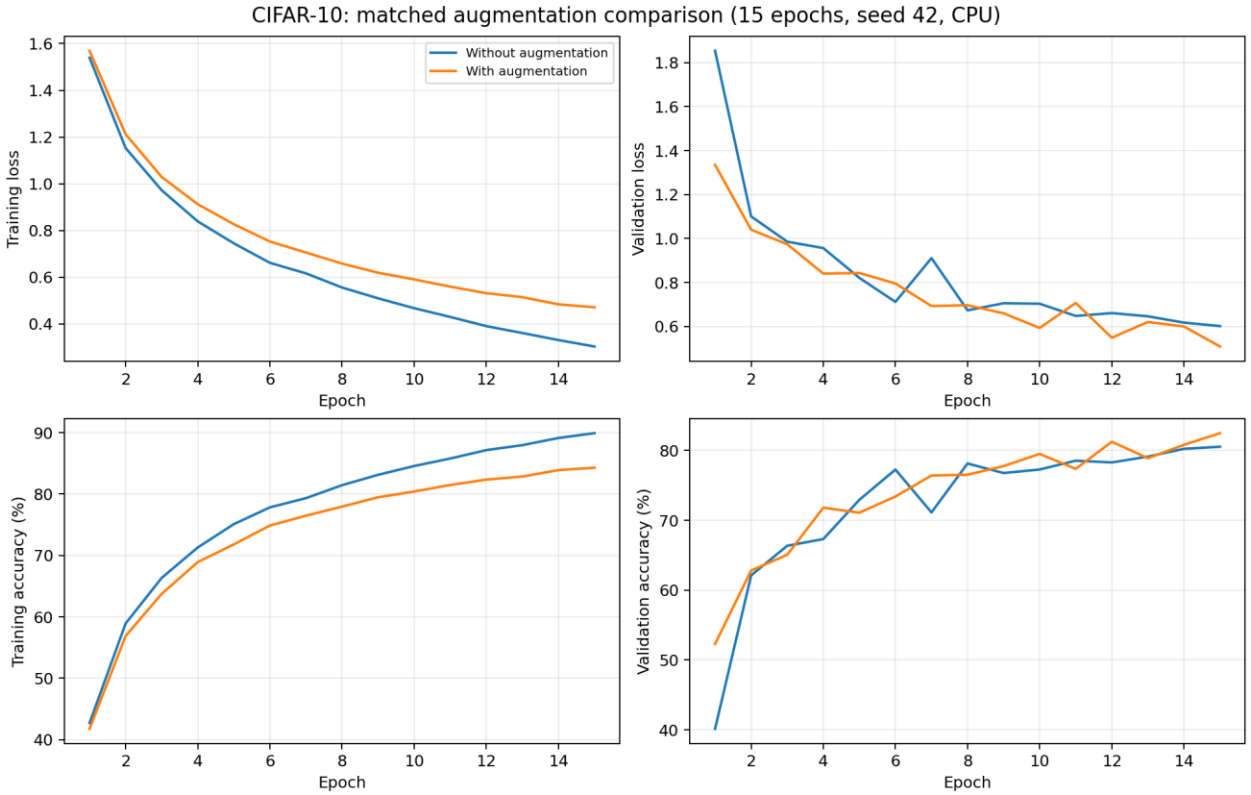


Figure 5: CIFAR-10 augmentation study: training and validation loss (top) and accuracy (bottom), 15 epochs.

Generalization gap reduction - Yes. On CIFAR-10 the unaugmented run reached 89.91% training and 80.53% validation accuracy (Δ = 9.38 percent point); the augmented run reached 84.28% and 82.47% (Δ = 1.81 percent point), a reduction of 7.57 pp (80.7%).

Validation metric improvement - Yes. Validation accuracy rose from 80.53% to 82.47% and Macro F1 from 80.54% to 82.68%. Each transformation changes the pixels but not the label, random crops shift the object by up to four pixels so a filter that relied on an object appearing at one canonical position stops reducing the loss because convolution shares weights across positions, the network is pushed to recognize the pattern wherever it appears. Horizontal flips add mirrored views and colour jitter randomizes brightness, contrast and saturation discouraging reliance on absolute colour values. Since the network almost never sees the same image twice memorizing individual samples no longer pays off and the filters must encode features shared by the whole class. Figure 5 shows the signature of this regularization: the augmented run has higher training loss (0.4708 versus 0.3028) but lower validation loss (0.5087 versus 0.6014).

On Tiny ImageNet the effect was marginal at 10 epochs: 29.60% with augmentation versus 30.63% without. Both runs have negative gaps and their curves are still rising at epoch 10 (look Figure 6) so the network is underfitting and had not yet assimilated the augmented distribution, the transformations are not too severe since the identical pipeline improved CIFAR-10. Meaning longer schedule is needed before augmentation's regularizing benefit appears on this dataset.

Dual-dataset benchmark: Tiny ImageNet

```
python train.py --model alexnet --norm bn --dataset tiny_imagenet --epochs 10 --save-model
checkpoints/alexnet_tiny_imagenet.pth --plot-file plots/loss_tiny_imagenet.png
```

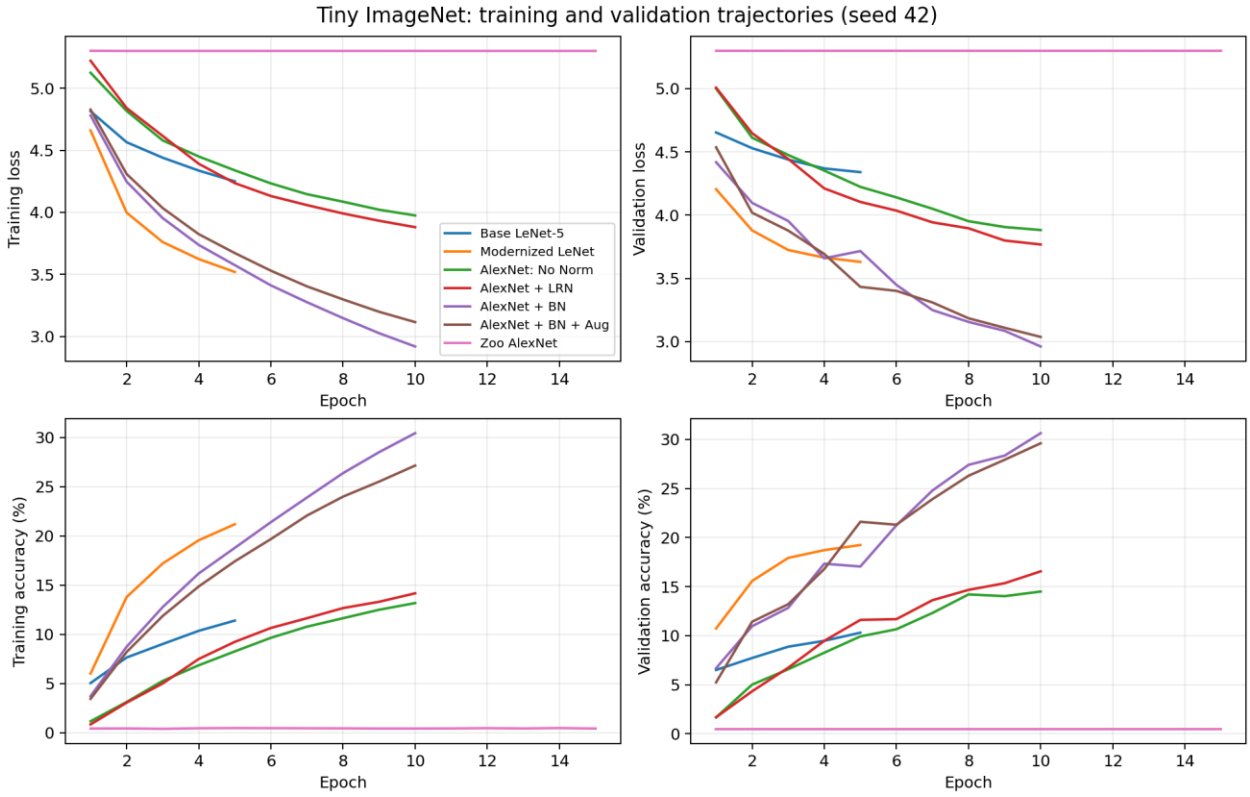


Figure 6: Tiny ImageNet training and validation loss (top) and accuracy (bottom) for all seven configurations (seed 42).

Generalization above random chance - Yes. AlexNet + BN + Aug reached 29.60% validation accuracy and 27.48% Macro F1, 59.2 times the 0.5% random-guess baseline. Every configuration except the Zoo AlexNet learned well above chance.

Cross-dataset performance comparison. The same model scores 82.47% / 82.68% on CIFAR-10 but 29.60% / 27.48% on Tiny ImageNet, about 53 lower. Tiny ImageNet is far harder for three reasons. First It has 200 classes instead of 10 many of them fine grained and visually similar (for example several dog breeds), so chance falls from 10% to 0.5% and decision boundaries are much closer together. Second It has 500 training images per class instead of 5,000 which is ten times less evidence per class. Third its 64 × 64 images are natural ImageNet photographs with clutter, varied scale and off centre objects, whereas CIFAR-10 images are 32 × 32 thumbnails of a single centred object upsampled to 64 × 64. Macro F1 is 2.12 percentage points below accuracy on Tiny ImageNet but 0.21 percentage points above it on CIFAR-10 which shows that performance is also less even across classes.

Overfitting trajectory - No. The Tiny ImageNet validation loss did not diverge or plateau earlier than CIFAR-10's. With augmentation it decreased in every epoch (4.5360 to 3.0377) and without augmentation it fell from 4.4183 to 2.9621 with one temporary rise at epoch 5 (Figure 6). Both final gaps are negative (−2.46 and −0.20): after 10 epochs the network is still underfitting the 100,000 image, 200 class problem. CIFAR-10 shows the opposite, it shows that its unaugmented BN validation loss rose at epochs 7, 9, and 12 while training loss kept falling, and Δ widened to 9.38 by epoch 15 (Figure 5). CIFAR-10's ten classes with 5,000 images each are fitted quickly, so overfitting appears there first Tiny ImageNet would need a much longer schedule before its validation loss turns upward.

Extended ablation exploration table

Each row changes one factor against its baseline. Val is the change in validation accuracy and Gap the change in Δ (negative means less overfitting). E5 is the chosen custom experiment: the Adam learning rate is lowered from 0.001 to 0.0003 for AlexNet + BN + Aug on CIFAR-10 (15 epochs, RTX 3070) with seeds 42 and 123 because the BN runs at 0.001 showed validation loss spikes.

Table 3.5: Extended ablation exploration (Δ comparisons)

Experiment	Change	Setting	Val Acc (%)	ΔVal (pp)	Gap Δ (pp)	ΔGap (pp)
E1 Normalization	None → LRN	CIFAR-10, 10 ep, seed 42	73.62 → 71.82	-1.80	2.57 → 2.66	+0.09
E2 Normalization	None → BatchNorm	CIFAR-10, 10 ep, seed 42	73.62 → 77.27	+3.65	2.57 → 7.29	+4.72
E3 Normalization	None → BatchNorm	Tiny ImageNet, 10 ep, seed 42	14.49 → 30.63	+16.14	-1.32 → -0.20	+1.12
E4 Augmentation	Off → On	CIFAR-10 BN, 15 ep, seed 42	80.53 → 82.47	+1.94	9.38 → 1.81	-7.57
E5 Learning rate (custom)	0.001 → 0.0003	CIFAR-10 BN + Aug, 15 ep, mean of seeds 42 and 123	81.73 → 84.36	+2.63	2.79 → 2.64	-0.15

Custom experiment commands (run for SEED = 42 and 123):

```
python train.py --model alexnet --norm bn --dataset cifar10 --epochs 15 --lr 0.001 --seed SEED --save-model
checkpoints/alexnet_bn_aug_gpu_seedSEED.pth --plot-file plots/loss_alexnet_bn_aug_gpu_seedSEED.png

python train.py --model alexnet --norm bn --dataset cifar10 --epochs 15 --lr 0.0003 --seed SEED --save-model
checkpoints/alexnet_bn_aug_lr0003_gpu_seedSEED.pth --plot-file plots/loss_alexnet_bn_aug_lr0003_gpu_seedSEED.png
```

Multi seed evaluation (Mean ± Std)

Statistics use the final epoch of each run: mean $\bar{x} = \Sigma x_i/n$ and sample standard deviation $s = \text{sqrt}(\Sigma(x_i - \bar{x})^2/(n-1))$ with $n = 2$ seeds.

Table 3.6a: Individual runs

Learning rate	Seed	Train Acc (%)	Val Acc (%)	Val Macro F1 (%)	Gap Δ (pp)	Mean Epoch Time (s)
0.001	42	84.52	81.22	80.99	3.30	28.58
0.001	123	84.52	82.24	82.27	2.28	28.28
0.0003	42	87.26	84.15	84.08	3.11	28.50
0.0003	123	86.74	84.57	84.62	2.17	28.88

Table 3.6b: Mean ± Std across seeds

Learning rate	n	Train Acc (%)	Val Acc (%)	Val Macro F1 (%)	Gap Δ (pp)	Mean Epoch Time (s)
0.001	2	84.52 ± 0.00	81.73 ± 0.72	81.63 ± 0.91	2.79 ± 0.72	28.43 ± 0.21
0.0003	2	87.00 ± 0.37	84.36 ± 0.30	84.35 ± 0.38	2.64 ± 0.66	28.69 ± 0.27

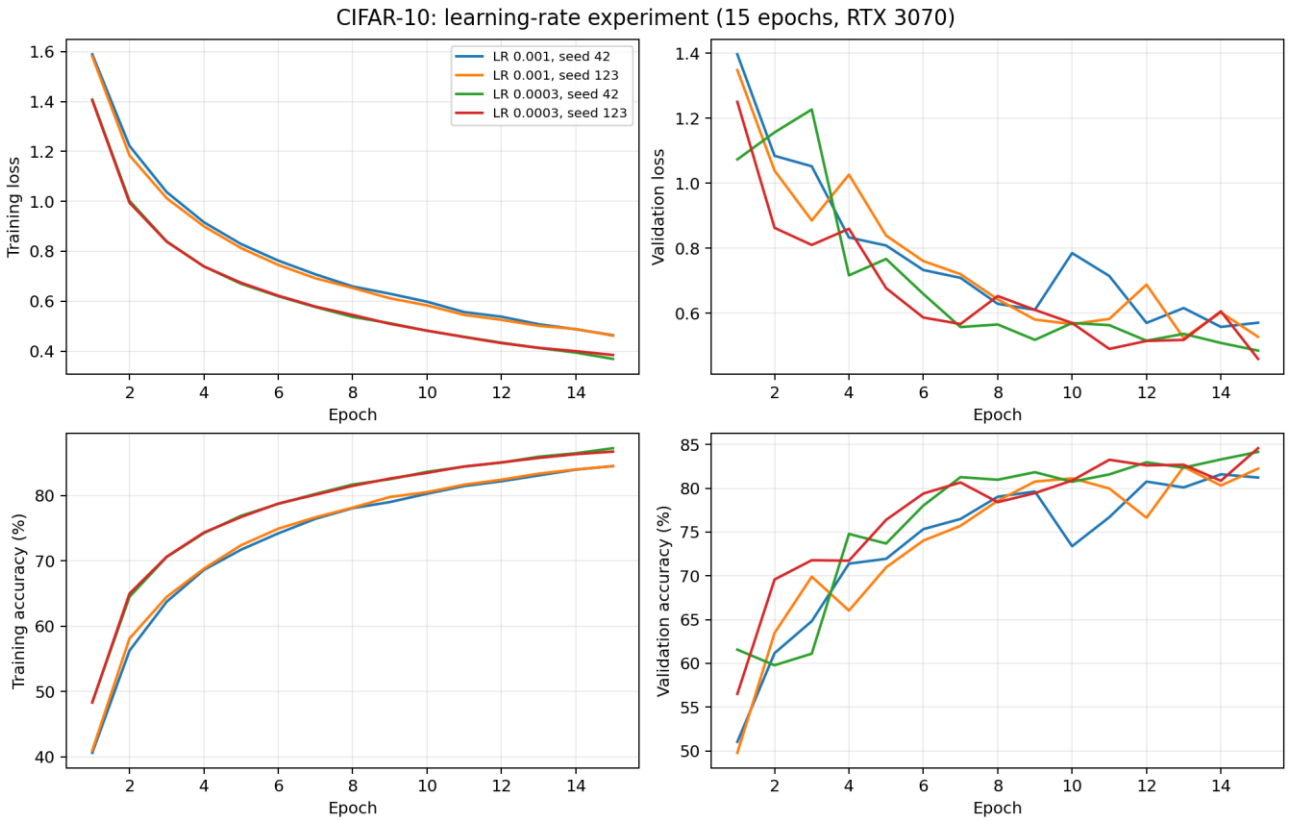


Figure 7: Custom learning-rate experiment: training and validation loss (top) and accuracy (bottom) for the four GPU runs.

3.7 demo_transforms.py output

Sample output of python demo_transforms.py (RTX 3070 PC), showing the data type, shape/layout, value range, and memory location produced by each pipeline:

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :   12.55 ms  ( 79,654 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   20.47 ms  ( 48,843 img/s) + Host-to-Device Copy required
- PyTorch (GPU Tensors)   :  240.85 ms  (  4,152 img/s) [ZERO PCIe Transfer Overhead]
================================================================================
```

Class balance and feature distributions (utils/eda.py)

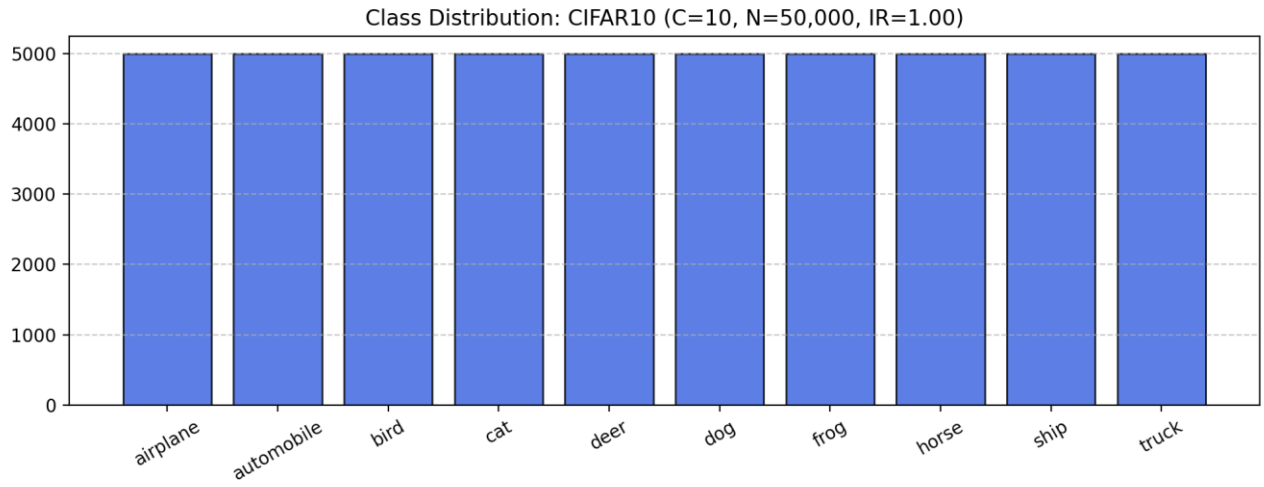


Figure 8: CIFAR-10 training class distribution (5,000 images in each class).

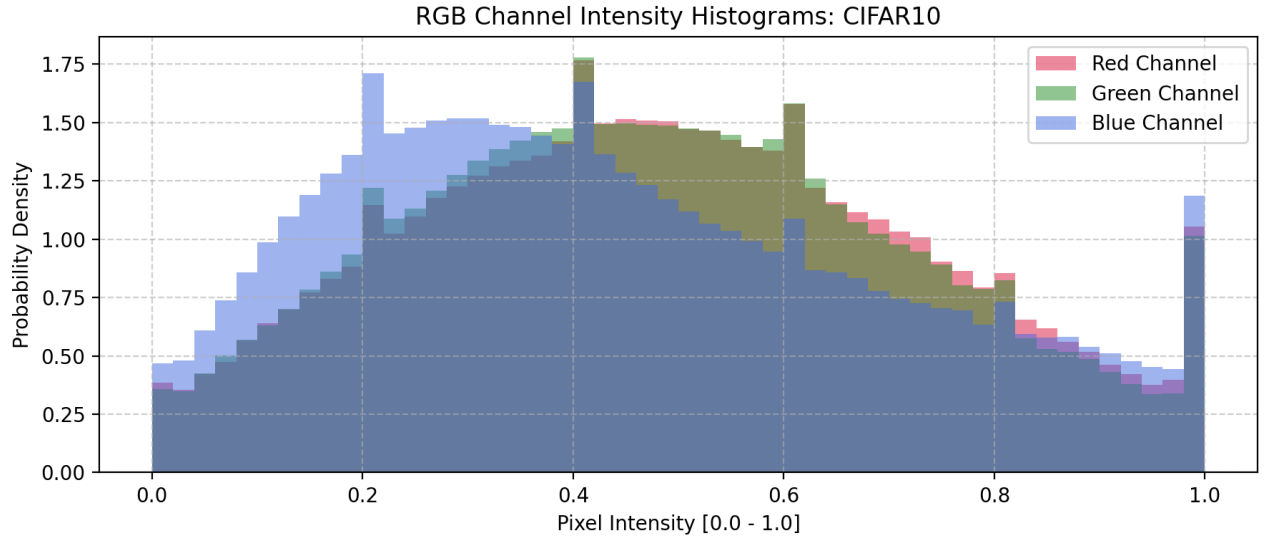


Figure 9: CIFAR-10 RGB channel intensity histograms.

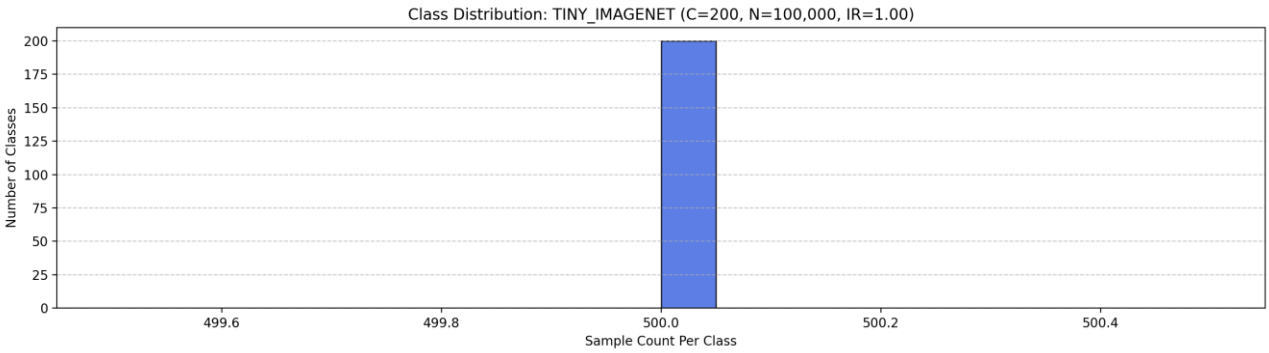


Figure 10: Tiny ImageNet training class distribution (500 images in each class).

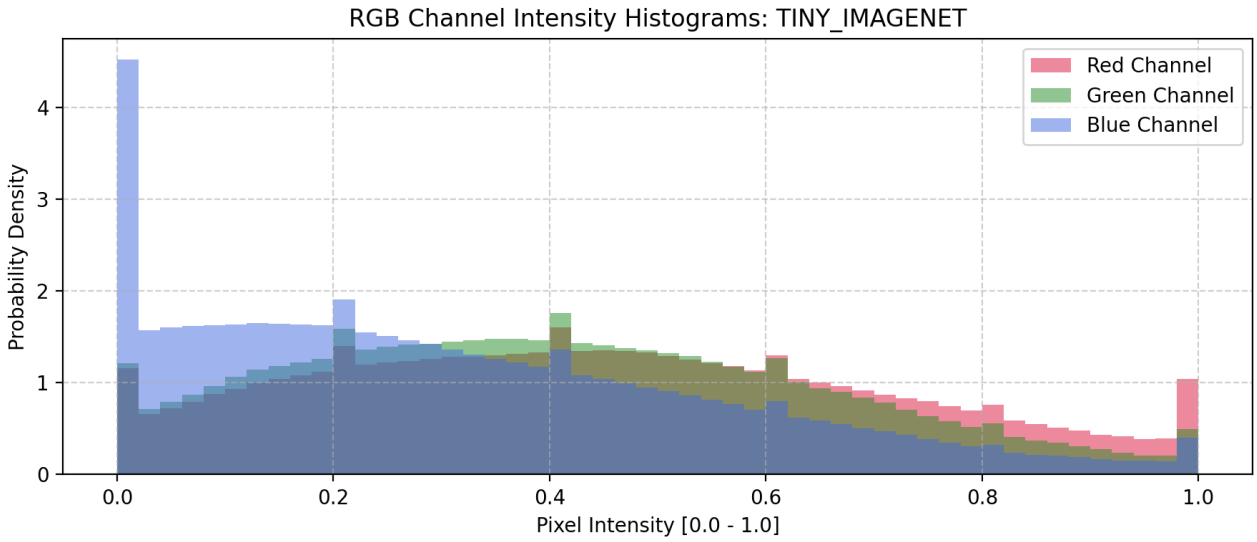


Figure 11: Tiny ImageNet RGB channel intensity histograms.

compare_models.py terminal output

```
> python compare_models.py --dataset cifar10 --device cuda
Device        : cuda (NVIDIA GeForce RTX 3070)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.40         | 2528.4             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.44         | 2254.3             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 0.86         | 1168.1             |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 1.36         | 735.2              |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 1.01         | 987.2              |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 1.86         | 536.8              |
> python compare_models.py --dataset tiny_imagenet --device cpu
Device        : cpu (CPU)
Dataset       : TINY_IMAGENET (200 classes)
Input Shape   : (1, 3, 64, 64)
| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.355      | 3.60     | 7.20      | 0.59         | 1687.8             |
| 2. LeNet-Modern (ReLU+Pool) | 0.462      | 4.61     | 9.22      | 0.75         | 1330.5             |
| 3. AlexNet-64 (None)        | 6.804      | 218.54   | 437.08    | 6.48         | 154.4              |
| 4. AlexNet-64 (LRN)         | 6.804      | 218.54   | 437.08    | 10.49        | 95.3               |
| 5. AlexNet-64 (BatchNorm)   | 6.806      | 219.18   | 438.36    | 6.98         | 143.3              |
| 6. Torchvision Zoo AlexNet  | 57.823     | 94.88    | 189.76    | 15.48        | 64.6               |
```

4. Technical Analysis & Discussion:

This section explains the performance differences between the seven configurations by following the progression from LeNet-5 to AlexNet one design change at a time. The handout's Step 2–4 questions are answered with their results in Section 3 here the focus is on why each change helped or did not and on what the results show about accuracy, generalization and cost together.

Base LeNet-5 to modernized LeNet: activation and pooling

Modernizing LeNet gave the largest gain for the smallest cost in the study. CIFAR-10 validation accuracy rose from 53.16% to 64.55% (+11.39 pp) and Macro F1 from 53.11% to 63.99%, and on Tiny ImageNet accuracy nearly doubled from 10.30% to 19.23%. The modernized model also converged faster: it reached 57.32% by epoch 2, already above the base model's final accuracy after five epochs, and its validation loss ended at 1.0058 versus 1.3176 (Figure 3).

Three mechanisms explain the difference. First, Tanh saturates: its derivative $1 - \tanh^2(x)$ approaches zero for large inputs, so the error signal shrinks as it passes back through each layer, whereas ReLU has a derivative of exactly 1 for every active unit and passes gradients through undiminished. Second, average pooling blends a strong feature response with its weaker neighbours, while max pooling keeps only the strongest response in each window, so a detected edge or texture survives downsampling at full strength; the overlapping 3 × 3 windows also make that selection less sensitive to the exact position of the feature. Third, padding keeps the border information that unpadded convolutions discard, and the larger 15 × 15 final map gives the classifier 3,600 instead of 2,704 inputs and a 19-pixel instead of 16-pixel receptive field. These changes were made together, so the gain cannot be assigned to one of them alone.

The cost was modest: parameters rose 31.8%, FLOPs rose 28% (7.17 to 9.19 M), mean CPU epoch time rose 13.6% (38.05 to 43.24 s), and GPU latency rose only from 0.40 to 0.44 ms. Neither model had plateaued after five epochs, since both validation losses were still falling in every epoch.

Modernized LeNet to AlexNet-64: depth and capacity

Moving to the five-stage AlexNet-64 without normalization raised CIFAR-10 accuracy by a further 9.07 pp to 73.62%. The deeper network builds a hierarchy of features, from edges to parts to objects, with a 97-pixel receptive field that covers the whole image, and its 64–384 channels per layer give far more capacity than LeNet's 6 and 16. This came at a large cost: 47.5 times the FLOPs (436.70 versus 9.19 M) and about 16 times the CPU epoch time.

On Tiny ImageNet the same model scored only 14.49%, below the modernized LeNet's 19.23%. Depth alone did not help on the harder task because the unnormalized 6.6 M-parameter network optimizes slowly: its accuracy was still rising steadily at epoch 10 (Figure 2), whereas the small LeNet fits 200 classes quickly. Greater capacity only becomes useful once the network can be trained efficiently, which is exactly what normalization provides.

Normalization: why BatchNorm helped and LRN did not

BatchNorm was the most effective single change to AlexNet-64. It raised CIFAR-10 accuracy from 73.62% to 77.27% and more than doubled Tiny ImageNet accuracy, from 14.49% to 30.63%. The larger gain on Tiny ImageNet confirms that the unnormalized model was limited by optimization,

not capacity: by keeping each layer's inputs at a stable mean and variance and smoothing the loss landscape, BatchNorm let Adam make steady progress at learning rate 0.001, and the BN model had the lowest training loss at every epoch.

BatchNorm is not a regularizer here, however. On CIFAR-10 it widened the generalization gap from 2.57 to 7.29 pp because it improved training accuracy (76.19% to 84.56%) faster than validation accuracy, and its validation loss was the noisiest of the three variants (Figure 4). The faster fitting is what made data augmentation necessary in the next step.

LRN gave no consistent benefit: it lowered CIFAR-10 accuracy by 1.80 pp and raised Tiny ImageNet accuracy by 2.06 percentage points, differences of the same order as the variation between runs. LRN only rescales each activation by the activity of neighbouring channels and does nothing to stabilize the distribution of layer inputs, so it does not address the optimization problem that limits the deep model. It was also the most expensive variant at inference (1.36 ms versus 0.86 ms on GPU), because its cross-channel sum of squares and fractional power is memory-bound. This matches the observation in the VGG paper that LRN adds cost without improving accuracy, and explains why BatchNorm replaced it.

Data augmentation: dependence on the training regime

Augmentation produced the best CIFAR-10 model (82.47%, 82.68% Macro F1) but slightly lowered Tiny ImageNet accuracy (29.60% versus 30.63%). The difference is explained by where each model sits on the bias–variance trade-off. On CIFAR-10 the BN model was overfitting: by epoch 15 without augmentation its training accuracy was 89.91% against 80.53% on validation. Augmentation turns each training image into many label-preserving variants, which removes the benefit of memorization and cut the gap to 1.81 percentage points. On Tiny ImageNet the model was still underfitting after ten epochs, with training accuracy below validation accuracy in both runs. A regularizer cannot help a model that has not yet fitted the training data; it only makes the training task harder. Augmentation should begin to pay off on Tiny ImageNet once a longer schedule brings the model into the overfitting regime.

Zoo AlexNet versus AlexNet-64: matching the architecture to the input

The torchvision AlexNet has 8.6 times as many parameters as AlexNet-64 (57.0 M versus 6.6 M) yet scored 6.13 pp lower on CIFAR-10 (67.49% versus 73.62%) and stayed at the 0.50% chance level on Tiny ImageNet for all 15 epochs, with a final validation loss of 5.2985, essentially ln 200 = 5.2983, the loss of a uniform prediction.

The cause is the mismatch between its design resolution and the 64 × 64 input. Its 11 × 11 stride-4 first layer and three pooling stages, designed to reduce a 224 × 224 image to 6 × 6, reduce a 64 × 64 image to a single 1 × 1 position. AdaptiveAvgPool2d then copies that single 256-value vector 36 times, so the 37.8 M-parameter first dense layer receives no spatial layout at all, while AlexNet-64's gentler 5 × 5 stride-2 first layer keeps a 3 × 3 map. With no normalization, a very large randomly initialized classifier, and this degraded input, the 200-class model never left its initial uniform prediction. The Zoo model also shows that parameters, FLOPs, and speed measure different things: it has the most parameters but fewer FLOPs than AlexNet-64 at this resolution (188 versus 437 M),

yet it is the slowest model at inference (1.86 ms) because moving its 54.6 M classifier weights through memory dominates the run time.

Custom experiment and run-to-run variability

Lowering the Adam learning rate from 0.001 to 0.0003 raised mean CIFAR-10 accuracy from 81.73 ± 0.72% to 84.36 ± 0.30% and improved both seeds, while the generalization gap barely changed. The gain is therefore better optimization rather than less overfitting: smaller steps let the BatchNorm network settle into a better minimum and reduce the mismatch between the batch statistics used in training and the running statistics used at evaluation, which caused the validation spikes seen at 0.001. The 2.63 percentage points improvement is larger than the seed to seed standard deviation, although two seeds are too few for a formal significance test.

The same configuration at learning rate 0.001 and seed 42 scored 82.47% on the CPU and 81.22% on the GPU, a 1.25 pp difference caused only by different hardware and numerical libraries. Together with the 0.72 percentage points seed standard deviation, this shows that differences of 1–2 pp between single-seed rows, such as LRN versus no normalization, are within normal run-to-run variation and should not be over-interpreted.

Accuracy versus computational cost

Across all configurations, parameter count was a poor predictor of performance: the most accurate model on CIFAR-10 (AlexNet-64 with BN and augmentation, 6.6 M parameters) beat the largest model (Zoo, 57.0 M) by 15 percentage. Convolutions dominate computation while dense layers dominate parameters, so FLOPs track the feature extractor and memory tracks the classifier. In terms of value per unit of compute, modernizing LeNet was the most efficient, BatchNorm was nearly free and augmentation and the learning-rate change cost nothing at inference. The large step to AlexNet-64 bought the highest accuracy but required about 48 times the arithmetic of the modernized LeNet.

These comparisons are limited by the experimental design: epoch budgets differ between rows (5, 10, or 15), most rows use a single seed, and some runs were trained on a GPU while most used a CPU under varying background load, so epoch times indicate direction rather than exact cost.